

Architecture sweep: GPT versus nGPT

A controlled comparison of the baseline LayerNorm GPT against [nGPT](#), and a second sweep that pins down a geometry bug in our simplified nGPT residual – found, and fixed in one line.

This is a **report**: it reads results the experiment already produced and renders them. The experiment itself is `experiment.py`, an importable `main(ctx)` DAG (one data-prep step, then a fifteen-cell GPU sweep) run from the CLI on Modal L4s (the `train` role binds the GPU + timeout):

```
bin/mini run docs/gpt-sweep/experiment.py --app modal
```

On completion the experiment publishes its val-loss curves to the project store under a stable name, and this report resolves them by that name when it renders – so the data lives in the durable store (the HF bucket), not in Git.

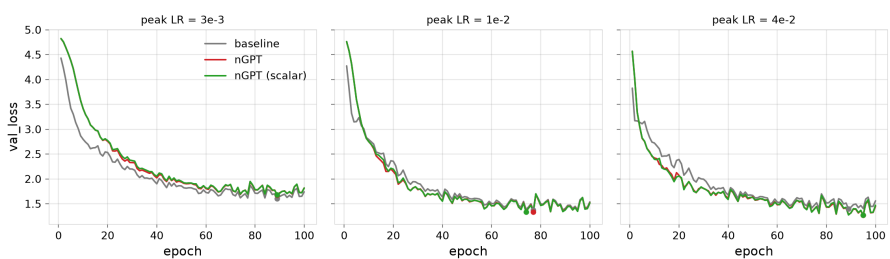
Best run: nGPT (scalar) at peak LR 4e-2 – val_loss **1.27** over 9 architecture×LR cells.

Does normalization help? nGPT versus the LayerNorm baseline

nGPT added a fair amount of machinery, so it's worth checking how it compares against the baseline. The two architectures don't want the same learning rate: normalization rescales the effective gradients, which shifts nGPT's useful LR band upward. To isolate the architecture, we sweep both across the same three peak learning rates, with the same warmup-then-cosine schedule, on the same data. The only difference within each panel is LayerNorm versus the hypersphere.

- **baseline** — standard pre-norm transformer (LayerNorm + additive residual).
- **nGPT** — normalized transformer as published (per-channel eigen learning rates).
- **nGPT (scalar)** — a single learnable scalar gate per sub-module instead of per-channel weights.

Both nGPT arms use the corrected normalized-LERP residual (see the next section); at this width the correction is invisible, so this is a clean architecture comparison.



Best (minimum) validation loss per cell:

peak LR	baseline	nGPT	nGPT (scalar)
3e-3	1.60	1.67	1.68
1e-2	1.37	1.34	1.33
4e-2	1.39	1.27	1.27

Two things stand out. **nGPT needs a higher learning rate to pay off** — at the lowest rate it trails the baseline, but given enough LR it reaches a slightly lower loss. And the **scalar-gate simplification matches the full per-channel variant** almost exactly: the per-channel granularity buys nothing at this scale.

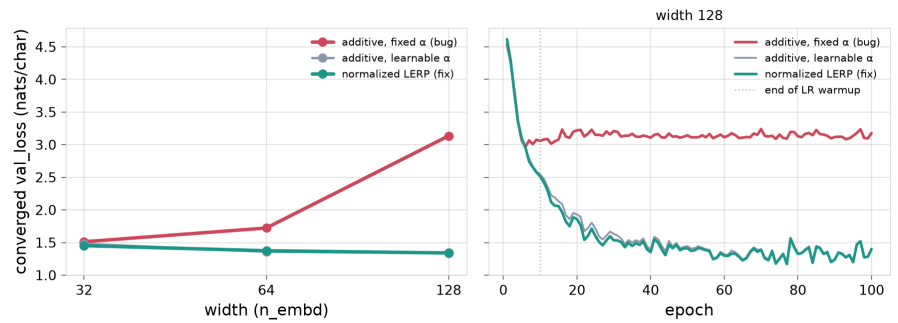
The paper's headline "trains several times faster" and its stability advantages don't reproduce at this toy scale — the LayerNorm baseline never threatens to diverge — so at this scale nGPT is a small but real improvement in final loss. The load-bearing piece is the scalar gate on the *normalized* residual (see the [model notes](#)); the rest of nGPT's machinery is optional here.

The residual bug: a width-gated failure the gate was hiding

The nGPT residual should step toward the sub-module's **normalized** output, $h \leftarrow \text{Norm}(h + \alpha \cdot (\text{Norm}(\text{sub}(h)) - h))$, which makes α a true interpolation fraction. We had simplified it to a raw additive step, $h \leftarrow \text{Norm}(h + \alpha \cdot \text{sub}(h))$, on the assumption that $\text{sub}(h)$ has $\text{norm} \approx 1$. It doesn't: the MLP scales its pre-activations by a $\sqrt{n_{\text{embd}}}$ baseline (to keep GELU in range), so $\|\text{MLP}(h)\| \propto \sqrt{n_{\text{embd}}}$. The *effective* rotation is $\alpha \cdot \|\text{sub}(h)\|$, which **grows with width** – a fixed α can't control it.

Whether that bites depends on the gate. Our α is *learnable* and reparametrized by $\sqrt{n_{\text{embd}}}$, so it can shrink as width grows and quietly absorb the error – which is why the sweep above (all width 32) saw nothing. To separate the geometry from the gate, we sweep width $\{32, 64, 128\}$ (with $n_{\text{ff}} = 4 \cdot n_{\text{embd}}$, so the MLP grows too) at depth 12 and LR 1e-2, in three recipes: the additive step with a **fixed** $\alpha = 1/n_{\text{layer}}$ (exposes the bug), the additive step with our **learnable** α (masks it), and the **normalized LERP** (removes it).

A fixed gate exposes the bug; the fix removes it. With a fixed step size the additive residual falls apart as width grows – converged loss climbs to **3.14** nats/char at width 128. The normalized-LERP fix reaches **1.34** there, flat across all three widths (spread just 0.11 nats/char). Same learning rate, same everything else; one line of geometry.



With a **fixed** step the additive residual is fine narrow and breaks wide: $1.51 \rightarrow 1.72 \rightarrow 3.14$ nats/char across widths 32 / 64 / 128. At width 128 it never really trains – the hidden state over-rotates through every layer until the token's identity is lost. That's the failure the normalized residual exists to prevent.

Two things rescue it, for different reasons. Our **learnable** α masks the bug ($1.48 \rightarrow 1.36 \rightarrow 1.34$): reparametrized by $\sqrt{n_{\text{embd}}}$, it shrinks as width grows and happens to cancel the $\|\text{sub}(h)\|$ growth – but that's the gate laundering a geometry error, not a correct residual. The **normalized LERP** removes the error at the source ($1.45 \rightarrow 1.37 \rightarrow 1.34$): normalizing $\text{sub}(h)$ makes α a true interpolation fraction, so the per-layer rotation is $\approx \alpha$ regardless of width or gate.

This is why the correction matters even though the architecture sweep couldn't see it: the scalar-gate nGPT we ship is only safe because *something* controls the residual geometry, and the normalized LERP does it by construction rather than by luck. The additive forms survive as `config.normalize_sublayer=False` (with `learnable_alpha=False` for the fixed gate), kept solely to reproduce this failure; the normalized LERP is the default. See the [model notes](#) for the geometry.

References

Loshchilov, I., Hsieh, C.-P., Sun, S., & Ginsburg, B. (2024). nGPT:

Normalized transformer with representation learning on the

hypersphere. *arXiv*. <https://arxiv.org/abs/2410.01131>

Karpathy, A. (2022). nanoGPT [Computer software]. GitHub.

<https://github.com/karpathy/nanoGPT>